

WarehouseInventoryMonitor

Sistem de Monitorizare a Stocurilor unui Depozit

Tip document	Documentatie tehnica
Autor	Baranai Tudor
Grupa	3122B
Disciplina	Programare Orientata pe Obiecte (C++)
Data	3 iunie 2026

1. Introducere si obiectiv

WarehouseInventoryMonitor este o aplicatie de consola scrisa in C++ (standard C++17) care gestioneaza stocul de produse al unui depozit. Operatorul poate adauga si elimina produse, le poate reaprovisiona (restock) sau vinde, poate cauta, filtra si sorta produsele, si poate genera rapoarte pentru produsele ramase sub pragul de alerta.

Interfata este organizata ca un catalog de produse, inspirat de magazinele online: fiecare produs are categorie, pret, cantitate disponibila, status de stoc si detalii specifice tipului sau. Pe langa produse, aplicatia gestioneaza furnizori (asociati cu produsele pe care le livreaza) si un istoric complet, cu marca de timp, al fiecarei miscari de stoc.

Toate datele sunt persistente: la pornire aplicatia incarca automat datele din trei fisiere CSV, iar dupa fiecare operatie importanta le salveaza inapoi, astfel incat starea depozitului se pastreaza intre rulari.

2. Concepte de Programare Orientata pe Obiecte

Proiectul ilustreaza principalele concepte POO cerute de tema:

Concept	Unde este folosit
Clase si obiecte	Produs, ProdusElectronic, ProdusMobilier, Depozit, Furnizor, MiscareStoc, IstoricTranzactii, Tranzactie<TP>
Incapsulare	Atribute private, accesate prin metode publice controlate
Mostenire	ProdusElectronic / ProdusMobilier extind Produs; MiscareIntrare / MiscareIesire extind MiscareStoc
Polimorfism	Metode virtuale: getTip(), getDetaliiSpecifice(), clone(), getDeltaStoc()
Clasa abstracta	MiscareStoc, cu metode pur virtuale, stocata polimorfic prin shared_ptr<MiscareStoc>
Supraincarcare operatori	Produs::operator+= si Produs::operator-=
Template	Tranzactie<Intrare> si Tranzactie<Iesire>
Exceptii	ID duplicat, produs lipsa, cantitate invalida, stoc insuficient, CSV invalid
STL	unordered_map, vector, shared_ptr, chrono
Persistenta	Produse, furnizori si istoric salvate in trei fisiere CSV separate

3. Arhitectura aplicatiei

Aplicatia este structurata in jurul clasei Depozit, care joaca rolul de fatada asupra intregului sistem. Depozit detine trei colectii:

- catalogul de produse: unordered_map<int, shared_ptr<Produs>>, cheia fiind ID-ul produsului;

- lista de furnizori: `unordered_map<int, Furnizor>`;
- istoricul de tranzactii: un obiect `IstoricTranzactii` care pastreaza miscarile de stoc.

Stocarea produselor prin `shared_ptr<Produs>` (pointer catre clasa de baza) este esentiala: ea permite ca produse standard, electronice si de mobilier sa coexiste in aceeaasi colectie, iar metodele virtuale sa fie apelate polimorfic, fara ca depozitul sa cunoasca tipul concret al fiecarui produs.

Fisierul `main.cpp` contine bucla de meniu si stratul de interactiune cu utilizatorul, delegand toata logica de business catre `Depozit`. Astfel, interfata de consola este complet separata de model.

4. Structura proiectului

Director / fisier	Continut
<code>src/</code>	Codul sursa C++ (clase si main)
<code>tests/</code>	Teste unitare simple (<code>test_inventory.cpp</code>)
<code>docs/</code>	Documentatia proiectului si notele UML
<code>data/</code>	Fisierele CSV folosite ca stocare persistenta
<code>Makefile</code>	Comenzi de compilare si testare
<code>CMakeLists.txt</code>	Configurarea alternativa de build (CMake)
<code>README.md</code>	Prezentare generala si instructiuni

5. Ierarhia de clase

5.1. Produs (clasa de baza)

`Produs` reprezinta un produs standard de depozit. Atributele sunt private: `id`, `nume`, `categorie`, `cantitate`, `pret` si `pragAlerta`.

Metode importante:

- `esteSubPrag()` - verifica daca produsul trebuie reaprovizionat (cantitate sub prag);
- `getStatusStoc()` - returneaza `In stoc`, `Stoc redus` sau `Indisponibil`;
- `getTip()` - metoda virtuala pentru tipul produsului;
- `getDetaliiSpecifice()` - metoda virtuala pentru detaliile afisate polimorfic;
- `clone()` - metoda virtuala pentru copiere polimorfica;
- `operator+=` / `operator-=` - adauga, respectiv scade cantitate din stoc.

5.2. ProdusElectronic si ProdusMobilier

Cele doua clase mostenesc `Produs` si adauga un atribut specific, suprascriind metodele virtuale pentru a oferi un comportament propriu:

- ProdusElectronic adauga garantieLuni; getTip() returneaza Electronic, iar getDetaliiSpecifice() afiseaza durata garantiei;
- ProdusMobilier adauga material; getTip() returneaza Mobilier, iar getDetaliiSpecifice() afiseaza materialul.

5.3. Depozit

Gestioneaza catalogul de produse, furnizorii si istoricul. Metode reprezentative:

- adaugaProdus(), eliminaProdus();
- restockProdus(), vindeProdus() - actualizeaza stocul si inregistreaza miscarea in istoric;
- cautaProduseDupaNume(), filtreazaDupaCategorie(), sorteazaDupaPret();
- raportProduseSubPrag(), sugereazaReaprovizionare();
- adaugaFurnizor(), asociazaProdusCuFurnizor(), sugestiiReaprovizionarePeFurnizor();
- incarca... / salveaza... - persistenta produselor, furnizorilor si istoricului.

5.4. Furnizor

Reprezinta un furnizor asociat cu unul sau mai multe produse. Atribute: id, nume, contact si produseFurnizate (lista de ID-uri de produse). Asocierile sunt gestionate prin adaugaProdusFurnizat() si eliminaProdusFurnizat(), iar furnizeazaProdusul() verifica daca un produs este asociat. La eliminarea unui produs din depozit, acesta este automat dezasociat de la toti furnizorii care il refereau.

5.5. MiscareStoc, MiscareIntrare, MiscareIesire

MiscareStoc este o clasa abstracta care modeleaza o miscare de stoc (produs, cantitate, timestamp, observatii). Metodele getTip(), getDeltaStoc() si clone() sunt pur virtuale. MiscareIntrare reprezinta un restock - getDeltaStoc() intoarce +cantitate; MiscareIesire reprezinta o vanzare - getDeltaStoc() intoarce -cantitate.

5.6. IstoricTranzactii si Tranzactie<TP>

IstoricTranzactii este un container care pastreaza miscarile polimorfic, prin vector<shared_ptr<MiscareStoc>>. Oferă înregistrarea intrărilor și ieșirilor, filtrarea pe produs, accesul la ultimele N miscări și persistența în data/istoric.csv.

Tranzactie<TP> este o clasa template care modeleaza o tranzactie de stoc; parametrul de tip poate fi Intrare (restock) sau Iesire (vanzare), demonstrand reutilizarea logicii prin templates.

6. Diagrama claselor (relatii)

```

Produs <|---- ProdusElectronic      (+ garantieLuni)
Produs <|---- ProdusMobilier        (+ material)

MiscareStoc (abstracta)
  <|---- MiscareIntrare              getDeltaStoc() -> +cantitate
  <|---- MiscareIesire               getDeltaStoc() -> -cantitate

Depozit 1 ----- * Produs           (prin shared_ptr<Produs>)
Depozit 1 ----- * Furnizor
Depozit 1 ----- 1 IstoricTranzactii
IstoricTranzactii 1 ----- * MiscareStoc (prin shared_ptr<MiscareStoc>)
Furnizor 1 ----- * Produs           (prin ID-uri)

```

7. Persistenta datelor

Aplicatia foloseste trei fisiere CSV separate in directorul data/. Toate sunt incarcate automat la pornire (daca exista) si rescrise dupa fiecare operatie care modifica starea. Separatorul de campuri este caracterul punct si virgula (;).

7.1. data/produse.csv - catalogul de produse

```

tip;id;nume;categorie;cantitate;pret;pragAlerta;detaliuExtra
Electronic;101;Laptop Lenovo IdeaPad;Laptopuri;12;2499.99;3;24
Mobilier;201;Dulap alb cu doua usi;Dulapuri;4;699.50;2;PAL melaminat
Standard;401;Bec LED economic;Accesorii;60;14.99;20;

```

Campul detaliuExtra are inteles diferit in functie de tip: numarul de luni de garantie pentru Electronic, materialul pentru Mobilier si este gol pentru Standard.

7.2. data/furnizori.csv - furnizorii si produsele asociate

```

id;nume;contact;produseIds (separate cu virgula)
10;ElectroDist SRL;contact@electrodist.ro;101,102,103,104
20;AudioVideo Pro;office@avpro.ro;105,106,108

```

7.3. data/istoric.csv - istoricul tranzactiilor

Fiecare restock si fiecare vanzare este inregistrata cu o marca de timp reala.

```

tip;timestamp;produsId;cantitate;observatii
Intrare;2026-05-04 09:15:22;101;10;comanda lunara furnizor
Iesire;2026-05-05 11:40:08;101;3;vanzare online

```

8. Meniul aplicatiei

Meniul de consola este grupat in patru sectiuni: produse, catalog, furnizori si istoric de tranzactii.

```

[Produse]
  1. Adauga produs           5. Afiseaza toate produsele
  2. Elimina produs        6. Raport produse sub prag
  3. Restock produs        7. Sugestii re aprovizionare
  4. Vanzare produs

[Catalog]
  8. Cauta produs dupa nume 11. Sorteaza pret descrescator
  9. Filtreaza dupa categorie 12. Salveaza tot
 10. Sorteaza pret crescator 13. Reincarca tot din fisiere

[Furnizori]
 14. Adauga furnizor        17. Produse pentru un furnizor
 15. Listeaza furnizori    18. Re aprovizionare pe furnizor
 16. Asociaza produs furnizor 19. Elimina furnizor

[Istoric tranzactii]
 20. Istoric complet       22. Ultimele N miscari
 21. Istoric pentru un produs 23. Sterge istoric

0. Iesire

```

9. Functionalitati principale

Functionalitatea centrala este monitorizarea stocului printr-un catalog persistent:

- produsele raman stocate intre rulari, prin persistenta CSV;
- fiecare produs are un status de stoc vizibil (In stoc / Stoc redus / Indisponibil);
- produsele sub prag sunt raportate automat;
- sugestiile de re aprovizionare sunt sortate dupa cantitatea ramasa;
- re aprovizionarea poate fi grupata pe furnizor, pentru o singura comanda per furnizor;
- tipurile diferite de produse afiseaza detalii diferite, prin polimorfism.

10. Testare

Testele sunt implementate in tests/test_inventory.cpp folosind assert. Ele verifica: adaugarea produselor si ID-urile duplicate, operatorii de cantitate, statusul de stoc, mostenirea si polimorfismul, cautarea, filtrarea si sortarea dupa pret, exceptiile, raportul de produse sub prag, sortarea sugestiilor de re aprovizionare, clasa template pentru tranzactii, persistenta produselor in CSV, gestionarea furnizorilor (asociere, dez asociere automata la stergere, re aprovizionare pe furnizor) si istoricul de tranzactii (inregistrare, filtrare pe produs, ultimele N miscari, persistenta).

11. Compilare si rulare

Cerinte: un compilator C++17 (de exemplu g++) si make sau cmake.

Folosind make:

```
make          # compileaza aplicatia
make run      # ruleaza aplicatia
make test     # ruleaza testele
make clean    # sterge fisierele generate
```

Alternativ, folosind CMake:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build
```

Important: aplicatia foloseste cai relative (data/produse.csv). Ruleaz-o din radacina proiectului, astfel incat directorul de lucru sa contina folderul data/. In CLion, seteaza Working directory pe \$ProjectFileDir\$ ca sa se citeasca fisierele corecte.

12. Posibile imbunatatiri

- interfata grafica (folosind o biblioteca de UI);
- export al rapoartelor (produse sub prag, valoare totala stoc) in CSV sau HTML;
- raport cu valoarea totala a stocului pe categorii;
- o baza de date reala (de exemplu SQLite) in locul fisiereilor CSV.